ASSIGNMENT REPORT

CO2 – CRYPTOGRAPHY AND NETWORK SECURITY

Evaluate, Design and Defend a Secure Public-Key and Hybrid Cryptographic Strategy for a Multi-Site Healthcare Network

STUDENT DETAILS	
Name	Samarapaku Madhuri
Registration Number	192465036
Programme	B.E. – CSE (Cybersecurity)
Institution	Saveetha School of Engineering
Course Outcome	CO2
Total Marks	50

Submitted as an academic assignment report

ASSIGNMENT QUESTIONS

CO2 – Cryptography and Network Security

- Q1 – Symmetric Cryptosystem Selection and Modes of Operation (7 Marks)
- Q2 – Number-Theory Engine and Prime Selection for RSA (6 Marks)
- Q3 – RSA Encryption/Decryption, Weaknesses and Padding (7 Marks)
- Q4 – Diffie–Hellman Key Exchange, Authentication and Key Management (7 Marks)
- Q5 – Elliptic-Curve Cryptography and Public-Key Strategy (5 Marks)
- Q6 – Digital Signatures, Hybrid Encryption and Quantum Readiness (7 Marks)
- Q7 – Diagnose, Redesign and Validate the Complete Strategy (11 Marks)

The answers begin on the following page.

ABSTRACT

This report evaluates cryptographic methods for a healthcare network that connects a central hospital with remote clinics over public networks. The network must protect clinical records, establish session keys, authenticate communicating sites, preserve data integrity and transfer large files efficiently. The report examines symmetric algorithms and block-cipher modes, number-theory operations used in RSA, RSA weaknesses and padding, Diffie–Hellman key exchange, elliptic-curve cryptography, digital signatures, hybrid encryption and quantum-readiness. A revised architecture is proposed using authenticated ephemeral key exchange, authenticated symmetric encryption, digital signatures, secure key management and a planned transition to post-quantum cryptography.

Important experimental note: The worked values in this report are classroom-scale examples intended to demonstrate mathematical operations. They are not production parameters. Any execution time, attack duration or performance measurement must be collected by running the included code on the student's own system. No fabricated benchmark results are presented. Real patient information must never be used for these experiments.

CONTENTS

- Q1 – Symmetric Cryptosystem Selection and Modes of Operation
- Q2 – Number-Theory Engine and Prime Selection for RSA
- Q3 – RSA Encryption, Decryption, Weaknesses and Padding
- Q4 – Diffie–Hellman Key Exchange, Authentication and Key Management
- Q5 – Elliptic-Curve Cryptography and Public-Key Strategy
- Q6 – Digital Signatures, Hybrid Encryption and Quantum Readiness
- Q7 – Diagnose, Redesign and Validate the Complete Strategy
- Overall Conclusion and References

Q1 – SYMMETRIC CRYPTOSYSTEM SELECTION AND MODES OF OPERATION (7 MARKS)

1.1 Role of symmetric encryption

Symmetric encryption uses the same secret key, or a directly related key, for encryption and decryption. It is efficient for protecting large amounts of data such as diagnostic images, electronic health records and backup files. Its main operational challenge is secure key distribution: the sender and receiver must obtain the same secret key without exposing it to an attacker.

1.2 Comparison of DES, 3DES, Blowfish and RC5

Algorithm	Structure and block size	Key characteristics	Assessment
DES	Feistel block cipher; 64-bit block; 16 rounds.	56 effective key bits; fixed permutation and substitution structure.	Obsolete. Its key is too short against exhaustive search.
3DES	Applies DES three times, commonly in EDE form; 64-bit block.	Uses two or three DES keys; effective security is lower than a modern 128-bit cipher and performance is slow.	Legacy compatibility only. Small block size creates risks for large data volumes.
Blowfish	16-round Feistel block cipher; 64-bit block.	Variable key length from 32 to 448 bits; key setup is relatively expensive.	Historically useful, but its 64-bit block size makes it unsuitable for new high-volume deployments.
RC5	Parameterized block cipher; commonly described by word size, rounds and key length.	Flexible parameters; data-dependent rotations, addition and XOR operations.	Research/legacy option. Security depends strongly on selected parameters and implementation.

DES uses a 16-round Feistel network. Each round expands and mixes the right half with a round key, applies substitution boxes and permutes the result before combining it with the left half. 3DES repeats the DES transformation three times to increase resistance to brute-force attacks, but this comes with substantial performance cost. Blowfish uses key-dependent substitution boxes and a Feistel structure. RC5 is notable for its parameterized design and data-dependent rotations.

A larger key space alone does not prove that a cipher is secure. Block size, mode of operation, key schedule, implementation quality, nonce management, side-channel resistance and the amount of data encrypted under one key must also be considered.

1.3 Block ciphers and stream ciphers

A block cipher transforms fixed-size blocks of plaintext into ciphertext under a secret key. A mode of operation specifies how to use that primitive for messages longer than one block. A stream cipher generates a pseudorandom keystream and combines it with plaintext, commonly using XOR. Block ciphers in counter mode can also provide stream-like processing.

Feature	Block cipher	Stream cipher
Processing	Fixed-size blocks; mode handles longer messages.	Produces a keystream continuously or in segments.
Typical use	Files, databases, stored records and bulk network payloads.	Low-latency streams and applications where data arrives continuously.
Main caution	Mode and IV/nonce must be selected correctly.	Never reuse a keystream/nonce with the same key where the design forbids it.

1.4 Modes of operation

Mode	Operation	IV/nonce and security considerations	Parallelism / errors
ECB	Encrypts each block independently.	No IV. Equal plaintext blocks produce equal ciphertext blocks, revealing patterns. Not suitable for sensitive structured data.	Blocks can be parallelised; an error affects the corresponding block.
CBC	XOR each plaintext block with the previous ciphertext block before encryption.	Requires an unpredictable, unique IV. Padding is normally required. Must use authentication separately.	Encryption is sequential; decryption can be parallelised. Bit errors can affect two blocks.
CFB	Uses the previous ciphertext block as input to the block cipher; output is XORed with plaintext.	Requires a unique IV. Converts a block cipher into a self-synchronising stream-like mode.	Encryption is sequential; bit errors can affect the corresponding plaintext bit and following segment.
OFB	Repeatedly encrypts an internal feedback value to generate a keystream.	IV must never be reused with the same key. Reuse repeats the keystream.	Keystream can be generated in advance; bit errors generally affect only matching plaintext bits.
CTR	Encrypts successive counter values and XORs the result with plaintext.	Nonce/counter combinations must be unique under a key. Reuse repeats keystream and can expose relationships between plaintexts.	Highly parallel; bit errors affect corresponding bits only.

1.5 Why ECB reveals patterns and why nonce reuse is dangerous

Suppose a record contains repeated blocks, such as long runs of zero bytes or repeated field structures. Under ECB, each identical plaintext block encrypted with the same key produces an identical ciphertext block. An observer cannot directly read the plaintext, but can identify repetition and infer structure. CBC, CTR and authenticated-encryption modes avoid this specific ECB pattern leakage when correctly configured.

In CTR mode, ciphertext is computed as C = P XOR KS, where KS is the keystream generated from the key and nonce/counter. If a nonce is reused with the same key, two messages use the same keystream. Therefore C1 XOR C2 = P1 XOR P2. This relationship can reveal information about both plaintexts. For authenticated modes such as GCM, nonce reuse can additionally undermine authentication and permit forgeries.

1.6 Student experiment and recommendation

For a controlled experiment, create a test message containing repeated 16-byte blocks and encrypt it with ECB and CBC/CTR using a standard cryptographic library. Count repeated ciphertext blocks and compare them. Then encrypt two different messages under CTR using (a) distinct nonces and (b) a deliberately reused nonce. The reused-nonce test should be performed only with toy data.

```
# Demonstration of ECB pattern leakage using a standard library
# Install: python -m pip install pycryptodome
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad
```

```
key = b"0123456789abcdef"
block = b"PATIENT-RECORD!!"   # exactly 16 bytes
plaintext = block * 4

cipher = AES.new(key, AES.MODE_ECB)
ciphertext = cipher.encrypt(plaintext)

blocks = [ciphertext[i:i+16] for i in range(0, len(ciphertext), 16)]
print("Number of ciphertext blocks:", len(blocks))
print("Unique ciphertext blocks:", len(set(blocks)))
print("Repeated block detected:", len(set(blocks)) &lt; len(blocks))
```

The above is a demonstration of the mode property, not an implementation of DES, 3DES, Blowfish or RC5 from first principles. The assignment's implementation requirement should be completed by implementing or simulating at least three selected algorithms, verifying decryption, and recording the exact environment, input sizes, operation counts and measured elapsed time. If a library primitive is used, identify it clearly.

Recommendation: New healthcare deployments should use AES-256-GCM or ChaCha20-Poly1305 for authenticated encryption. These constructions provide confidentiality and integrity when keys and nonces are managed correctly. DES should not be used. 3DES and Blowfish should be treated as legacy-only because of their 64-bit block size and performance/security limitations. RC5 is not the preferred choice for a new standardised deployment.

Q2 – NUMBER-THEORY ENGINE AND PRIME SELECTION FOR RSA (6 MARKS)

2.1 Greatest common divisor

The greatest common divisor, gcd(a,b), is the largest positive integer that divides both a and b. The Euclidean algorithm repeatedly replaces (a,b) with (b, a mod b) until the remainder becomes zero. The last non-zero value is the GCD.

```
def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return abs(a)

print(gcd(48, 18))  # 6
```

2.2 Extended Euclidean algorithm and modular inverse

The extended Euclidean algorithm finds integers x and y such that ax + by = gcd(a,b). If gcd(a,m)=1, then ax ≡ 1 (mod m), so x modulo m is the modular inverse of a. RSA uses this operation to calculate the private exponent d as the inverse of e modulo φ(n).

```
def extended_gcd(a, b):
    if b == 0:
        return a, 1, 0
    g, x1, y1 = extended_gcd(b, a % b)
    return g, y1, x1 - (a // b) * y1

def mod_inverse(a, m):
```

```
    g, x, y = extended_gcd(a, m)
    if g != 1:
        raise ValueError("Inverse does not exist")
    return x % m

print(mod_inverse(3, 40))  # 27
```

For comparison, a trial-search inverse checks candidates one by one until (a × candidate) mod m equals 1. This is simple for classroom examples but can require many iterations. Extended Euclid reduces the problem through successive divisions and is efficient even for large integers.

2.3 Fast modular exponentiation

RSA repeatedly computes expressions of the form a^b mod n. Naive repeated multiplication performs approximately b multiplications. Square-and-multiply processes the exponent in binary and requires a number of modular multiplications proportional to the number of exponent bits.

```
def mod_pow(base, exponent, modulus):
    result = 1
    base %= modulus
    while exponent &gt; 0:
        if exponent % 2 == 1:
            result = (result * base) % modulus
        base = (base * base) % modulus
        exponent //= 2
    return result

print(mod_pow(7, 13, 55))  # 2
```

2.4 Euler's totient function and RSA condition

Euler's totient function φ(n) counts the positive integers from 1 through n that are relatively prime to n. For two distinct primes p and q, φ(pq) = (p−1)(q−1). RSA selects a public exponent e satisfying 1 < e < φ(n) and gcd(e,φ(n))=1. This condition ensures that e has a modular inverse d modulo φ(n), where ed ≡ 1 (mod φ(n)).

2.5 Primality testing

Method	Basic idea	Strength and limitation
Trial division	Test whether any integer from 2 through floor(sqrt(n)) divides n.	Deterministic and easy to understand, but slow for large candidates.
Fermat test	For a base a relatively prime to n, test whether a^(n−1) ≡ 1 (mod n).	Fast screening, but some composite numbers pass for many bases.
Miller–Rabin	Write n−1 = 2^s × d with d odd; test modular powers for selected bases.	Efficient probabilistic test. Repeating with independent bases makes the probability of a false prime result very small.

Carmichael numbers such as 561, 1105 and 1729 are composite numbers that can satisfy Fermat's congruence for every base relatively prime to the number. Therefore, a Fermat test alone is not a reliable primality proof. Miller–Rabin is a stronger practical test for generating large probable primes.

```
def is_probable_prime(n, bases=(2, 3, 5, 7, 11)):
    if n &amp;lt; 2:
        return False
    small = (2, 3, 5, 7, 11, 13, 17, 19, 23, 29)
    for p in small:
        if n == p:
            return True
        if n % p == 0:
            return False

    d, s = n - 1, 0
    while d % 2 == 0:
        d //= 2
        s += 1

    for a in bases:
        if a % n == 0:
            continue
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(s - 1):
            x = (x * x) % n
            if x == n - 1:
                break
        else:
            return False
    return True

for value in (97, 561, 1105, 1729):
    print(value, is_probable_prime(value))
```

2.6 Prime density and engineering decision

The prime number theorem estimates that the probability that a large integer near N is prime is approximately 1/ln(N). Thus, as candidate numbers become larger, the fraction of primes becomes smaller, and more candidates may need to be tested. RSA key generation therefore uses efficient probable-prime tests rather than trial division across the entire range.

Classroom-sized examples permit manual verification and trial division. At 1024, 2048 and 3072 bits, the arithmetic uses large integers and requires efficient big-number libraries, cryptographically secure random generation and carefully designed primality testing. A 1024-bit RSA modulus is no longer recommended for new systems; 2048 bits is a common minimum baseline, while 3072 bits offers a larger classical security margin. The organisation should follow current standards and migration guidance rather than selecting key size in isolation.

Experimental evidence to record: For each candidate, record the candidate value, algorithm, bases used, number of modular multiplications/divisions or loop iterations, result and elapsed time. Include 561, 1105 and 1729 to show why Fermat can misclassify Carmichael numbers. Do not report invented operation counts or timing values.

Q3 – RSA ENCRYPTION/DECRYPTION, WEAKNESSES AND PADDING (7 MARKS)

3.1 RSA key generation

RSA begins by selecting two distinct large primes p and q. It calculates n = pq and φ(n) = (p−1)(q−1). A public exponent e is selected such that gcd(e,φ(n))=1. The private exponent d is calculated as d = e ¹ mod φ(n). The public key is (n,e), and the private key contains d together with protected prime and CRT values.

3.2 Hand-verifiable example

Choose p=5 and q=11. Then n=55 and φ(n)=4×10=40. Choose e=3 because gcd(3,40)=1. The inverse of 3 modulo 40 is d=27 because 3×27=81 ≡ 1 (mod 40). The public key is (55,3), and the private exponent is 27.

Step	Calculation	Result
Key generation	p=5, q=11	n=55
Totient	φ(n)=(5−1)(11−1)	40
Public exponent	e=3; gcd(3,40)=1	Valid
Private exponent	d=3 ¹ mod 40	27
Encryption	C=7³ mod 55	13
Decryption	M=13²⁷ mod 55	7

The example verifies that encryption followed by decryption returns the original message. The primes and modulus are intentionally tiny and provide no security.

```
p, q = 5, 11
n = p * q
phi = (p - 1) * (q - 1)
e = 3
d = pow(e, -1, phi)

message = 7
ciphertext = pow(message, e, n)
recovered = pow(ciphertext, d, n)

print("Public key:", (n, e))
print("Private exponent:", d)
print("Ciphertext:", ciphertext)
print("Recovered message:", recovered)
```

3.3 Weaknesses of textbook RSA

Textbook RSA is deterministic: the same message encrypted with the same public key always produces the same ciphertext. This reveals equality of messages and enables dictionary-style guessing when the message space is small. RSA is also multiplicatively malleable: multiplying a ciphertext by a suitable value can produce a related plaintext after decryption. Small messages and poorly selected exponents can create additional risks, especially when messages are not padded and structured.

Messages must be represented as integers in the valid range $0 \leq M < n$. Invalid parameters, non-coprime exponents or incorrect key generation can prevent correct operation or weaken security. RSA key length does not compensate for weak randomness, shared primes, predictable primes, poor storage or side-channel leakage.

3.4 Factoring and recovery of the private key

If an attacker factors n into p and q, the attacker can calculate $\varphi(n)=(p-1)(q-1)$ and then compute $d=e^{-1} \bmod \varphi(n)$. This recovers the private exponent. Trial division is effective only for very small moduli. Fermat factorisation is particularly useful when p and q are close because it searches for a representation $n=a^2-b^2=(a-b)(a+b)$.

```
from math import isqrt

def fermat_factor(n):
    a = isqrt(n)
    if a * a &lt; n:
        a += 1
    while True:
        b2 = a * a - n
        b = isqrt(b2)
        if b * b == b2:
            return a - b, a + b
        a += 1

print(fermat_factor(10403))  # (101, 103), close factors
```

3.5 Padding and OAEP

Optimal Asymmetric Encryption Padding (OAEP) randomises RSA encryption by combining the message with a random seed and mask-generation operations before applying the RSA exponentiation. This prevents identical messages from producing identical ciphertexts and addresses important structural weaknesses of textbook RSA. RSA-OAEP should be used through a well-reviewed cryptographic library with an appropriate hash and mask-generation function. RSA should encrypt or wrap short key material, not large clinical files.

Test plan: Predict that textbook RSA encrypts repeated messages to identical ciphertexts, then test the same message twice. Test boundary inputs M=0, M=n−1 and M=n; the last value is outside the valid range. Compare with a properly implemented OAEP library call and confirm that repeated encryption is randomised. Use only toy keys and non-sensitive data.

Q4 – DIFFIE–HELLMAN KEY EXCHANGE, AUTHENTICATION AND KEY MANAGEMENT (7 MARKS)

4.1 Mathematical basis

Diffie–Hellman allows two parties to derive a shared secret over an observable network. The parties agree on a public prime p and generator g. Clinic A chooses private value a and sends $A=g^a$ mod p. Clinic B chooses private value b and sends $B=g^b$ mod p. A computes B^a mod p, while B computes A^b mod p. Both obtain the same value because $(g^b)^a \equiv (g^a)^b \equiv g^{ab} \pmod p$.

4.2 Worked example

Parameter	Value
Public prime p	23
Public generator g	5
Clinic A private value a	6
Clinic B private value b	15
A public value = 5^6 mod 23	8
B public value = 5^{15} mod 23	19
A shared value = 19^6 mod 23	2
B shared value = 8^{15} mod 23	2

Both clinics obtain the same shared value, 2. This small result is only for hand calculation. Real systems use standardised groups or elliptic-curve groups and derive a session key from the shared secret using a key derivation function.

4.3 Pseudocode and implementation

```
# Toy Diffie-Hellman example
p, g = 23, 5
a, b = 6, 15                # private values
A = pow(g, a, p)            # A sends this
B = pow(g, b, p)            # B sends this
key_A = pow(B, a, p)
key_B = pow(A, b, p)

print("Public values:", A, B)
print("Shared values:", key_A, key_B)
print("Match:", key_A == key_B)
```

4.4 Man-in-the-middle attack

Unauthenticated Diffie–Hellman provides secrecy against passive observation but does not prove who is at the other end of the exchange. An active attacker can intercept A's public value, substitute an attacker-controlled value to B, and perform a separate exchange with each clinic. The attacker then decrypts, reads or modifies traffic and re-encrypts it for the other party. Both clinics may believe that they share a key with each other when each actually shares a different key with the attacker.

The defence is to authenticate the key-exchange transcript. Digital signatures tied to trusted certificates can bind each party's identity to its ephemeral public key and relevant protocol parameters. Certificate validation must check the chain, validity period, intended identity and revocation status. Modern authenticated protocols also bind the negotiated transcript into key derivation and Finished/integrity checks.

4.5 Key-management policy

Lifecycle stage	Required control
Generation	Use an approved cryptographic random number generator; generate unique private values and keys.
Storage	Protect long-term private keys in a hardware security module or operating-system keystore; restrict access and audit use.
Distribution	Use authenticated key exchange and certificate validation; never send raw private keys.
Rotation	Use short-lived session keys and rotate long-term keys according to policy, risk and certificate validity.
Revocation	Maintain certificate revocation and incident-response processes; reject revoked or expired credentials.
Compromise recovery	Revoke affected credentials, isolate compromised endpoints, rotate keys and investigate exposure.
Session lifetime	Limit session duration and data volume; use forward-secret ephemeral key exchange where supported.

A larger toy prime can be used to compare legitimate modular exponentiation with a brute-force discrete-log search. Record the actual number of candidate exponents tested and elapsed time. The toy attack must not be extrapolated as a direct estimate of attacks on standard production groups.

Q5 – ELLIPTIC-CURVE CRYPTOGRAPHY AND PUBLIC-KEY STRATEGY (5 MARKS)

5.1 Mathematical basis of ECC

Elliptic-curve cryptography uses points on an elliptic curve over a finite field. A common curve form is $y^2 \equiv x^3 + ax + b \pmod{p}$, where p is prime and the curve parameters are selected so the curve is non-singular. The points, together with a point at infinity, form a group under elliptic-curve point addition. Repeated point addition is called scalar multiplication.

A private key is a randomly selected integer d. The public key is Q=dG, where G is a standard base point. Given d and G, calculating Q is efficient. Recovering d from Q and G is the elliptic-curve discrete logarithm problem, which is believed to be computationally difficult for properly selected curves and key sizes.

5.2 Toy point-operation illustration

For a classroom demonstration, define point addition over a small prime field and use scalar multiplication to compute a public point. The following code is illustrative only; it omits production-grade curve validation, constant-time arithmetic and secure parameter handling.

```
# Educational ECC point arithmetic over a small finite field.
# Curve: y^2 = x^3 + 2x + 2 (mod 17)
p, a = 17, 2
O = None

def inv(x):
    return pow(x % p, -1, p)

def add(P, Q):
    if P is O: return Q
    if Q is O: return P
    x1, y1 = P
    x2, y2 = Q
    if x1 == x2 and (y1 + y2) % p == 0:
        return O
    if P != Q:
        m = ((y2-y1) * inv(x2-x1)) % p
    else:
        m = ((3*x1*x1+a) * inv(2*y1)) % p
    x3 = (m*m-x1-x2) % p
    y3 = (m*(x1-x3)-y1) % p
    return (x3, y3)

def multiply(k, P):
    result, addend = O, P
    while k:
        if k &amp; 1:
            result = add(result, addend)
        addend = add(addend, addend)
        k &gt;&gt;= 1
    return result

G = (5, 1)  # point on this toy curve
private_value = 7
public_point = multiply(private_value, G)
print("Public point:", public_point)
```

5.3 RSA, finite-field DH and ECC comparison

Factor	RSA	Finite-field Diffie–Hellman	ECC
Main purpose	Encryption/signatures (with appropriate schemes).	Key agreement.	Key agreement and signatures, depending on scheme.
Security basis	Integer factorisation.	Discrete logarithm in a finite field.	Elliptic-curve discrete logarithm.
Key material	Larger keys for comparable classical security.	Large group parameters.	Smaller public keys for comparable classical security.
Deployment	Mature, widely supported; private-key operations can be costly.	Requires authenticated exchange and group validation.	Efficient bandwidth and computation for constrained devices; implementation must be carefully validated.

ECC can provide comparable classical security with substantially smaller public-key material than traditional RSA parameters. Smaller keys reduce bandwidth, storage and some computation costs, which is useful for remote clinics and constrained devices. This does not mean every ECC implementation is automatically secure: curve choice, randomness, validation, constant-time operations and private-key protection remain essential.

Recommendation: Use a modern authenticated elliptic-curve key-agreement protocol such as ephemeral ECDH within a standard secure transport protocol, combined with a KDF and authenticated symmetric encryption. Use standardised signature algorithms and validated libraries. Do not invent custom curves or deploy the toy curve shown above.

Q6 – DIGITAL SIGNATURES, HYBRID ENCRYPTION AND QUANTUM READINESS (7 MARKS)

6.1 Digital signatures: hash then sign

A digital signature provides evidence of message integrity and signer authenticity, and supports non-repudiation when identity and key custody are properly managed. The sender first calculates a cryptographic hash of the document. The sender signs the hash or a scheme-defined encoding of it using a private signing key. The receiver hashes the received document independently and verifies the signature using the corresponding public key. If the document changes, the hash changes and verification should fail.

6.2 RSA, DSA/DSS and ECDSA

Algorithm	Key and operation	Randomness and deployment notes
RSA signatures	Uses RSA public/private key pair with a secure signature encoding such as RSA-PSS.	PSS uses a salt and secure hashing. Key sizes are comparatively large; implementation and padding must be correct.
DSA/DSS	Discrete-log signature family; signature generation and verification use modular arithmetic.	Per-signature nonce must be unique and secret; nonce failure can expose the private key. Legacy use should follow current policy.
ECDSA	Elliptic-curve signature algorithm with compact keys and signatures.	Nonce generation is critical. Reusing or biasing the nonce can reveal the private key; deterministic nonce methods can reduce dependence on fresh randomness.

6.3 Hybrid encryption for large files

Public-key encryption is computationally expensive and is not designed to encrypt large files block by block. Hybrid encryption solves this by generating a random symmetric session key, encrypting the file with an authenticated symmetric cipher such as AES-256-GCM, and protecting or deriving the session key through an authenticated public-key mechanism. The receiver authenticates the sender and obtains the session key, then decrypts the bulk data efficiently.

A suitable high-level sequence is: (1) sender validates the recipient's certificate; (2) sender establishes an authenticated ephemeral key exchange; (3) both parties derive session keys with a KDF; (4) sender encrypts the file using AES-GCM with a unique nonce and associated metadata; (5) sender signs the required manifest or file hash; (6) receiver verifies the signature and authentication tag before accepting the content.

6.4 Tampering tests

Create a toy file and signature, then run three separate negative tests: change one byte of the encrypted file, alter one byte of the signature, and alter the wrapped or derived session-key material. The receiver should reject each modified item. Record the test input, modification, verification result and error handling. Never accept partially decrypted or unauthenticated clinical data.

6.5 Direct RSA versus hybrid encryption

Approach	Operation profile	Suitability for large files
Direct RSA encryption	Public-key operation is limited by the RSA modulus and padding overhead; the entire file cannot be encrypted as arbitrary-length RSA plaintext.	Not appropriate for large files.
Hybrid encryption	A small number of public-key/key-agreement operations plus symmetric encryption over the file bytes.	Recommended. Symmetric encryption scales efficiently with file size.

As file size grows, hybrid encryption adds approximately one symmetric encryption pass over the data and a small fixed amount of key-establishment/signature overhead. Direct RSA would require splitting data into small blocks, which is inefficient and introduces complexity and risk. Therefore, large diagnostic images and records should be protected with authenticated symmetric encryption.

6.6 Quantum cryptography and post-quantum readiness

Quantum key distribution (QKD) uses quantum states to establish or distribute key material. In idealised protocols, measurement by an eavesdropper disturbs quantum states and can increase the observed error rate, allowing the communicating parties to detect interference. QKD requires specialised equipment and an authenticated classical channel; it does not itself solve endpoint compromise, identity management, secure software updates or ordinary key lifecycle management.

A sufficiently capable fault-tolerant quantum computer running Shor's algorithm would threaten RSA and elliptic-curve cryptography because it can solve the relevant factoring and discrete-log problems efficiently in the theoretical model. Symmetric cryptography is affected differently; larger symmetric keys provide a stronger margin against known quantum search speedups.

The network should inventory cryptographic dependencies, prioritise long-lived sensitive records, design crypto-agile interfaces, test post-quantum algorithms and plan a controlled migration. NIST-standardised post-quantum options include ML-KEM for key encapsulation and ML-DSA for digital signatures. Hybrid classical/post-quantum key establishment can be considered during transition where standards and implementations support it. QKD may be evaluated for specialised links, but it is not a substitute for post-quantum migration or sound key management.

6.7 Deployment decision

The practical design is authenticated ephemeral key exchange, a KDF, AES-256-GCM or ChaCha20-Poly1305 for bulk data, and digital signatures for important records or manifests. This separates key establishment, bulk confidentiality and identity/integrity. It avoids using public-key encryption on every file block and supports forward secrecy when ephemeral keys are correctly used and erased.

Q7 – DIAGNOSE, REDESIGN AND VALIDATE THE COMPLETE STRATEGY (11 MARKS)

A. Diagnose the bottleneck or weakness (2 marks)

The principal weakness in the initial design is the possibility of unauthenticated key exchange combined with the inefficient use of public-key operations for bulk data. Unauthenticated Diffie–Hellman allows a man-in-the-middle to establish separate secrets with each clinic. Direct RSA encryption of large records also creates unnecessary computational overhead and is constrained by the RSA modulus and padding. Repeated use of a symmetric key with a reused nonce can undermine confidentiality and authentication.

B. Define the design objective (1 mark)

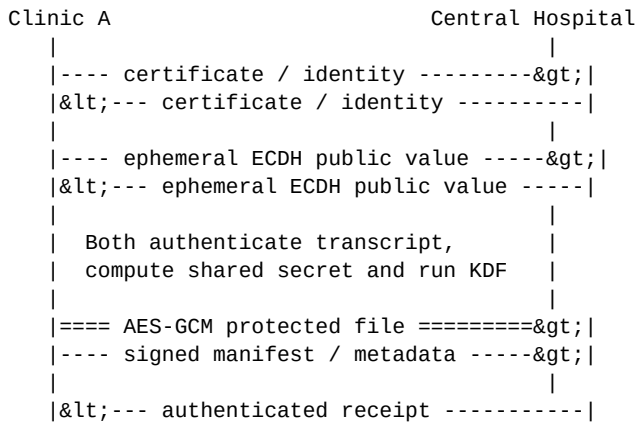
The measurable objective is to ensure that every clinic-to-hospital session is authenticated before protected data is accepted, use public-key operations only for authenticated key establishment and signatures, encrypt each bulk payload with an authenticated symmetric cipher, and reject any altered ciphertext or signature. Validation should record successful legitimate exchange, failed authentication/tampering tests, number of public-key operations per session and elapsed time for multiple file sizes.

C. Revised strategy (3 marks)

The redesigned architecture uses certificate-authenticated ephemeral ECDH to establish a shared secret, a KDF to derive separate directional session keys, AES-256-GCM or ChaCha20-Poly1305 to protect bulk records, and digital signatures for high-value messages and file manifests. Each message includes authenticated metadata such as sender identity, recipient identity, record identifier, protocol version and sequence number.

Stage	Operation	Security purpose
1. Identity validation	Validate the hospital/clinic certificate and intended peer identity.	Prevents unauthenticated peer substitution.
2. Ephemeral key exchange	Exchange ephemeral ECDH public values and authenticate the transcript with signatures/certificates.	Establishes a shared secret while resisting MITM.
3. Key derivation	Use a standard KDF to derive independent encryption and authentication/session keys.	Avoids using raw shared secret directly and separates key purposes.
4. Bulk protection	Encrypt each file using AES-256-GCM or ChaCha20-Poly1305 with unique nonces.	Provides confidentiality and integrity efficiently.
5. Document authenticity	Sign a manifest or hash containing file identity and relevant metadata.	Allows the receiver to verify the sender and detect changes.
6. Lifecycle controls	Rotate keys, expire sessions, revoke credentials and erase ephemeral secrets.	Limits exposure and supports compromise recovery.

Revised message sequence



The architecture reduces expensive public-key processing because public-key operations occur during session establishment and selected signature operations, not for every block of the clinical file. The additional requirements are certificate infrastructure, ephemeral key state, nonce tracking, KDF use, key rotation and secure endpoint storage. These controls introduce operational complexity but are justified by the sensitivity of healthcare records.

D. Integrate quantum readiness (1 mark)

RSA and ECC dependencies must be inventoried because sufficiently capable quantum computers threaten their mathematical assumptions. The network should design crypto-agile interfaces, track standards and vendor support, test ML-KEM for key establishment and ML-DSA for signatures, and plan migration based on data confidentiality lifetime and system replacement cycles. QKD may detect certain forms of eavesdropping during specialised key distribution, but it does not replace authentication, endpoint security, certificates, software security or post-quantum algorithms.

E. Predict, implement and validate (2 marks)

Prediction: compared with direct public-key protection of large files, the hybrid design will require fewer public-key operations and will scale more efficiently as file size increases. Authenticated key exchange should reject substituted public values when signatures and certificates are correctly verified. Authenticated encryption should reject modified ciphertext, nonce misuse should be prevented by the nonce policy, and the receiver should reject invalid signatures.

Validation should use student-created test data and record the following:

Test	Expected result	Evidence to record
Legitimate clinic-to-hospital exchange	Both sides derive matching session keys and decrypt the test file.	Key-match Boolean, plaintext hash comparison and elapsed time.

Test	Expected result	Evidence to record
MITM substitution	Certificate/transcript authentication fails; no clinical payload is accepted.	Rejected identity or signature and attack-step trace.
Ciphertext tampering	AEAD authentication fails.	Tampering location and rejection result.
Signature tampering	Signature verification fails.	Original and modified signature verification outputs.
Nonce policy	Duplicate nonce under one key is blocked by implementation/policy.	Nonce tracking log or test assertion.
Increasing file sizes	Symmetric encryption time grows with data volume; public-key setup remains a small fixed component per session.	File sizes, public-key operation count, symmetric operation count and measured time.

Do not fill the evidence table with guessed values. Run the program on the available computer, repeat each timing test several times, report the measurement method and hardware/software environment, and distinguish measured results from theoretical predictions.

F. Defend the final design (2 marks)

The final design is preferred because it combines authenticated identity, efficient bulk encryption, integrity checking, controlled key lifetimes and a migration path. DES and textbook RSA are rejected because they are unsuitable for modern sensitive deployments. 3DES and Blowfish are legacy options with small block sizes. Unauthenticated Diffie–Hellman is rejected because it is vulnerable to active substitution. Direct RSA encryption of large data is rejected because it is inefficient and not designed for bulk payloads. ECC is useful for compact authenticated key exchange, while symmetric authenticated encryption handles the file data.

Residual risks remain. A compromised clinic endpoint can expose plaintext before encryption or after decryption. Stolen signing keys can enable impersonation until revoked. Incorrect nonce management can undermine authenticated encryption. Certificate misconfiguration can block legitimate connections or admit the wrong peer. Quantum migration requires interoperability testing, vendor support, key-size planning and careful operational change management. These risks are addressed through endpoint hardening, hardware-backed key storage, monitoring, certificate lifecycle controls, secure randomness, key rotation, incident response and periodic cryptographic review.

OVERALL CONCLUSION

The healthcare network requires a layered cryptographic design rather than reliance on a single algorithm. Symmetric authenticated encryption is the appropriate method for large clinical records. Public-key techniques should establish session keys and authenticate identities, while digital signatures protect important documents and metadata. Number-theory operations support RSA but must be implemented with secure parameters and modern padding. Diffie–Hellman must be authenticated to prevent man-in-the-middle attacks. ECC provides compact public-key material for constrained sites, provided standard curves and validated implementations are used. Finally, the network should prepare for quantum threats through cryptographic inventory, crypto-agility and a planned transition to post-quantum standards.

REFERENCES

- William Stallings, Cryptography and Network Security: Principles and Practice, Pearson.
- Jonathan Katz and Yehuda Lindell, Introduction to Modern Cryptography, CRC Press.
- NIST, FIPS 197: Advanced Encryption Standard (AES).
- NIST, SP 800-56A: Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography.
- NIST, SP 800-57 Part 1: Recommendation for Key Management.
- NIST, FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM).
- NIST, FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA).
- RFC 8017, PKCS #1: RSA Cryptography Specifications Version 2.2.